ZOHO SETU PROJECT

Setu Project

Baseline kernel OS

Software Domain

Description

Create a minimal 64-bit microkernel with core components like USB drivers, memory management, and a graphical shell.

Project Type: Linux **Category:** Software Based

Atharva Bodade

Class: Third Year

Branch: Information Technology

Shri Sant Gajanan Maharaj College of Engineering, Shegaon

GitHub: https://github.com/UnsettledAverage73/zoho-os

Reference: https://kernel.org/

May 2026

Contents

Chapter 1

Vision & Problem Statement

The goal of this project is to develop a minimal, educational Linux-type kernel from scratch. It serves as a baseline for understanding low-level systems programming, memory management, and driver development.

1.1 The Limitations of Today

Modern kernels like Linux are incredibly complex, making it difficult for students and researchers to grasp the core fundamentals of how an OS interacts with hardware. This project aims to strip away the complexity to provide a clear, functional baseline.

1.2 Project Philosophy

Baseline kernel OS is built to be a minimal yet extensible system. This requires:

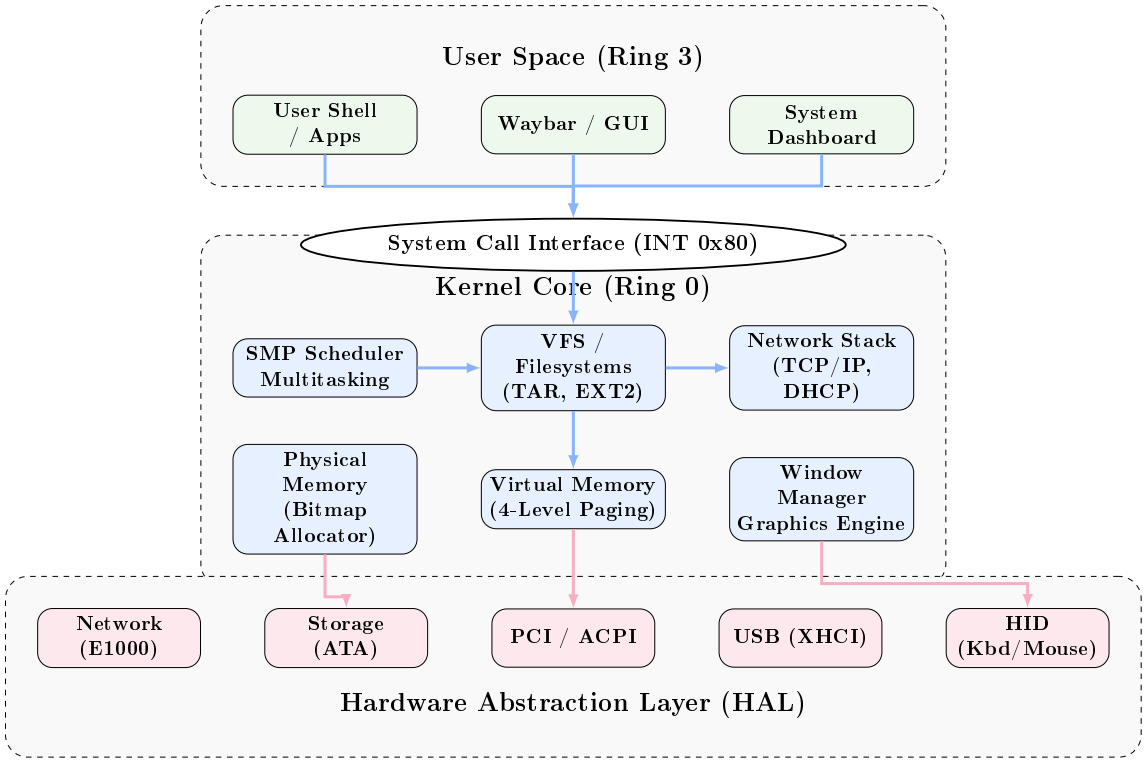
- **Educational Clarity:** Well-documented code that maps directly to architectural concepts.
- **Hardware Abstraction:** A clean interface between the kernel and the underlying x86_64 hardware.
- **Extensibility:** A modular design that allows for easy addition of new drivers and system calls.

Chapter 2

Architecture Overview

Baseline kernel OS implements a **Modular Monolithic Architecture**. While all core services run within the same address space in Ring 0 for performance, the system is logically divided into strictly defined layers and modules to ensure maintainability and structural clarity.

2.1 System Architecture Diagram



2.2 Structural Integrity

The architecture is designed around three primary pillars:

- **Isolation:** User-space applications are strictly isolated from the kernel and each other via 4-level paging and hardware-enforced protection rings.

- **Abstraction:** The VFS and HAL provide a consistent interface for higher-level systems, allowing hardware-specific details to be swapped or updated without affecting the core logic.
- **Observability:** Integrated logging (`klog`), statistics (`kstats`), and tracing (`ktrace`) are built into the kernel core to facilitate real-time debugging and performance monitoring.

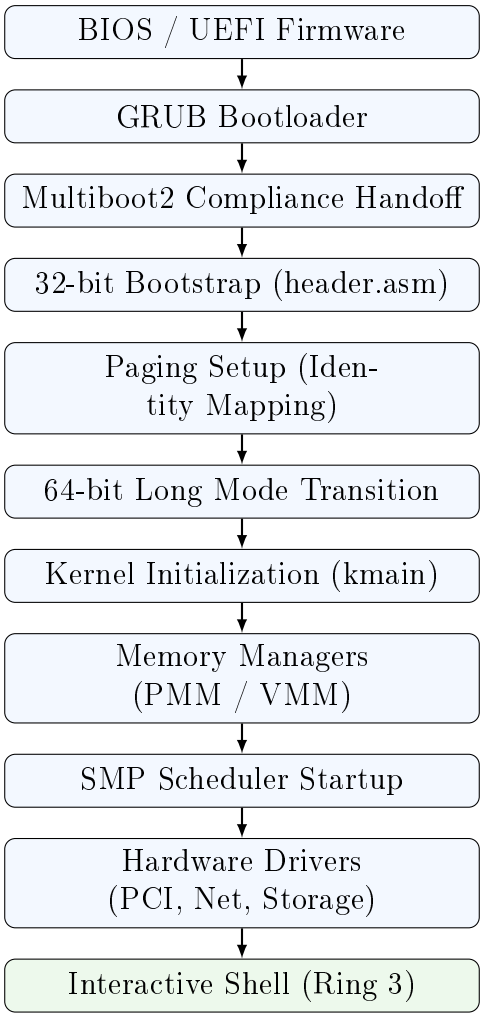
Chapter 3

Boot Process

The transition from a powered-off state to a running 64-bit environment involves a complex handoff between assembly and C.

3.1 System Lifecycle & Boot Sequence

The end-to-end initialization of Baseline kernel OS follows a strictly defined sequence of handoffs between hardware, firmware, and software layers.



3.2 Boot Sequence File Mapping

The following table maps the logical stages of the boot process to their respective implementation files in the source tree.

Stage	**Source File**	**Primary Responsibility**
Multiboot2 Header	`src/boot/header.asm`	GRUB handshake and 32-bit entry.
32-bit Bootstrap	`src/boot/main.asm`	Identity paging and GDT setup.
Long Mode Setup	`src/boot/main.asm`	CPU far jump and 64-bit transition.
Kernel Main	`src/kernel/main.c`	High-level C entry and initialization.
Memory Setup	`pmm.c / vmm.c`	Physical and virtual memory context.
SMP Handoff	`trampoline.asm`	AP (Application Processor) startup logic.

Table 3.1: Boot Process File Mapping

3.3 Key Milestones

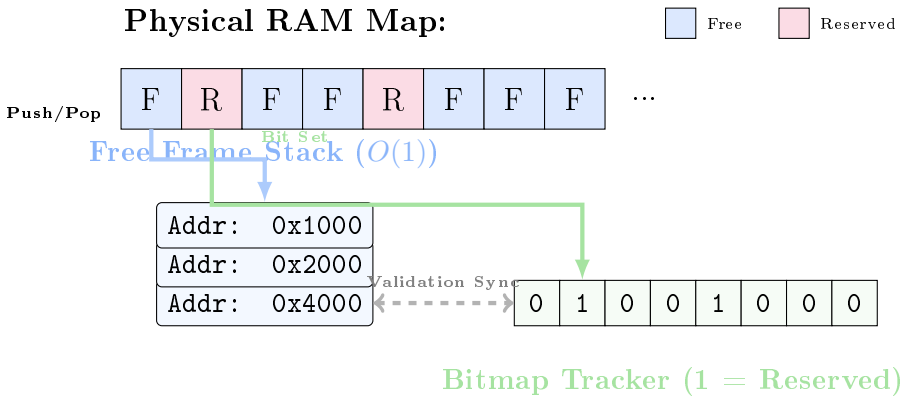
1. **Multiboot2 Header:** Allows GRUB to recognize the kernel.
2. **PAE Paging:** Necessary to address the 64-bit space.
3. **GDT Flush:** Loading the global descriptor table for code and data segments.

Chapter 4

Core Kernel Systems

4.1 Physical Memory Manager (PMM)

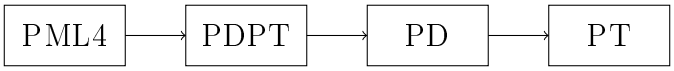
The PMM uses a high-performance hybrid architecture, combining a stack-based frame allocator for $O(1)$ allocation with a bitmap tracker for validation and redundancy.



PMM Hybrid Allocation Engine Architecture

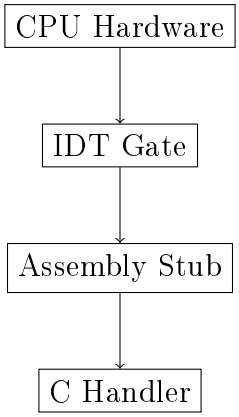
4.2 Virtual Memory Manager (VMM)

Baseline kernel OS implements a 4-level paging system.



4.3 Interrupt Handling & IDT

Baseline kernel OS manages hardware asynchronous events through a centralized Interrupt Descriptor Table (IDT).



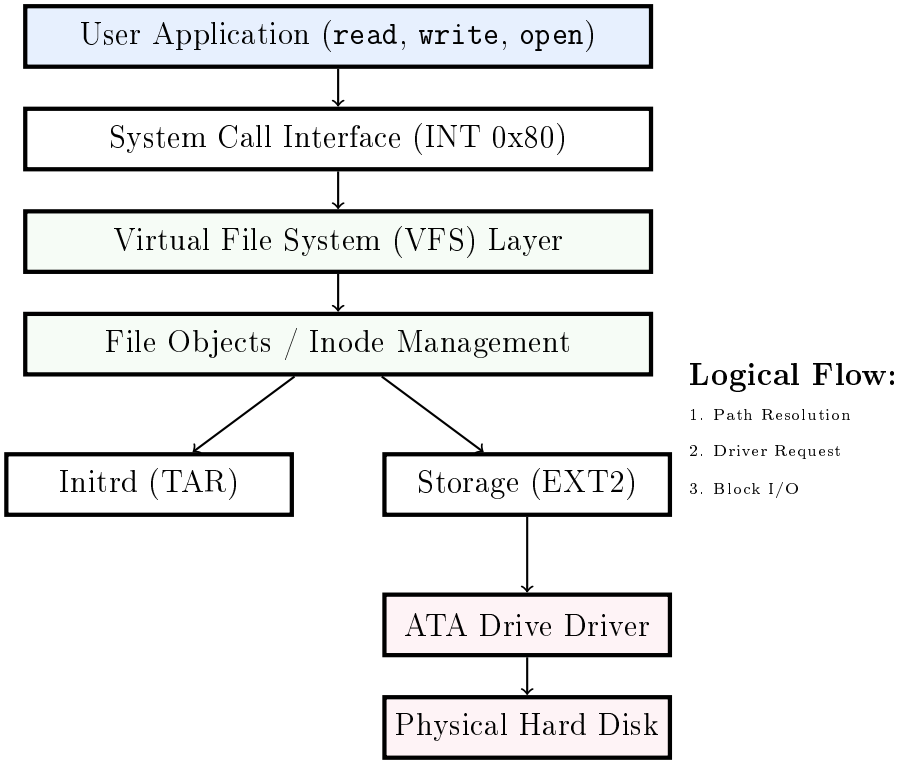
4.4 SMP Scheduler & Task Management

The scheduler implements a multicore round-robin algorithm with per-CPU runqueues and task stealing for dynamic load balancing.

SMP Multiprocessing & Load Balancing

4.5 File System Design & Data Flow

The Baseline kernel OS filesystem is designed around a strictly hierarchical abstraction layer. This allows the kernel to treat hardware devices, local storage, and memory-backed archives through a unified "File" interface.



4.6 Filesystem Implementation Mapping

The hierarchical design of the VFS is implemented across the following modules:

Layer	Source File	Implementation Role
VFS Abstraction	`src/kernel/vfs.c`	Path parsing and driver dispatching.
Initrd Driver	`src/kernel/tar.c`	Read-only access to disk-backed TAR archive.
Storage Driver	`src/kernel/ext2.c`	Ext2 block and inode management.
Block I/O	`src/kernel/ata.c`	Low-level ATA PIO mode disk driver.
Device Nodes	`src/kernel/tty.c`	Character device interface for `/dev/tty`.

Table 4.1: Filesystem Implementation Mapping

- **Disk-backed TAR:** Used for the initial ramdisk and system applications.
- **Device Nodes:** Support for `/dev/tty` and `/dev/serial`.

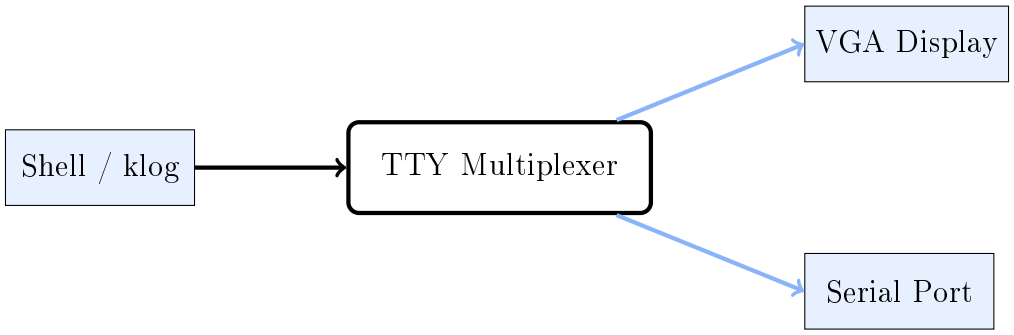
4.7 Observability (KStats & KTrace)

Baseline kernel OS includes built-in observability tools that track kernel performance from the earliest boot stages.

- **KStats:** Monitors physical memory usage, uptime, and active task counts.
- **KTrace:** Provides an event buffer for high-speed tracing of kernel execution paths, essential for debugging race conditions in SMP mode.

4.8 Terminal Abstraction (TTY)

The TTY layer provides a unified interface for text-based communication, multiplexing output between the local VGA framebuffer and the remote Serial (COM1) port.



TTY Output Multiplexing Flow

4.9 USB Support (XHCI)

Baseline kernel OS features an eXtensible Host Controller Interface (XHCI) driver foundation, enabling high-speed communication with USB 3.0 devices such as human interface devices (HID) and mass storage.

Chapter 5

Shell Commands & Interaction

Baseline kernel OS provides a comprehensive interactive shell for system management, debugging, and application execution. The following table details the primary commands available.

Command	**Description**	**Context**
`help`	Displays the list of all available commands.	General
`monitor`	Real-time CPU usage, Memory (MB used/total), and Task count.	Observability
`top`	Lists all active tasks with ID, CPU affinity, Ticks, and State.	Task Mgmt
`net`	Displays MAC/IP addresses and Packet RX/TX counts.	Networking
`ls`	Lists all files and directories in the current VFS path.	Storage
`cat <file>`	Reads and prints the contents of a file to the terminal.	Storage
`write <f> <t>`	Writes a string of text to a specified file.	Storage
`exec <file>`	Loads and executes a binary file from the VFS.	Process
`heap`	Detailed statistics of the Slab/Heap memory allocator.	Memory
`free`	Displays the total count of free physical memory frames.	Memory
`locks`	Shows acquisition and contention stats for kernel spinlocks.	Debugging
`stress_proc`	Initiates a "Process Storm" test to verify scheduler stability.	Stress Test
`clear`	Clears the terminal screen and resets the cursor.	General

Table 5.1: Primary Shell Commands

5.1 Interaction Examples

Baseline kernel OS shell supports various commands for deep-system inspection and control.

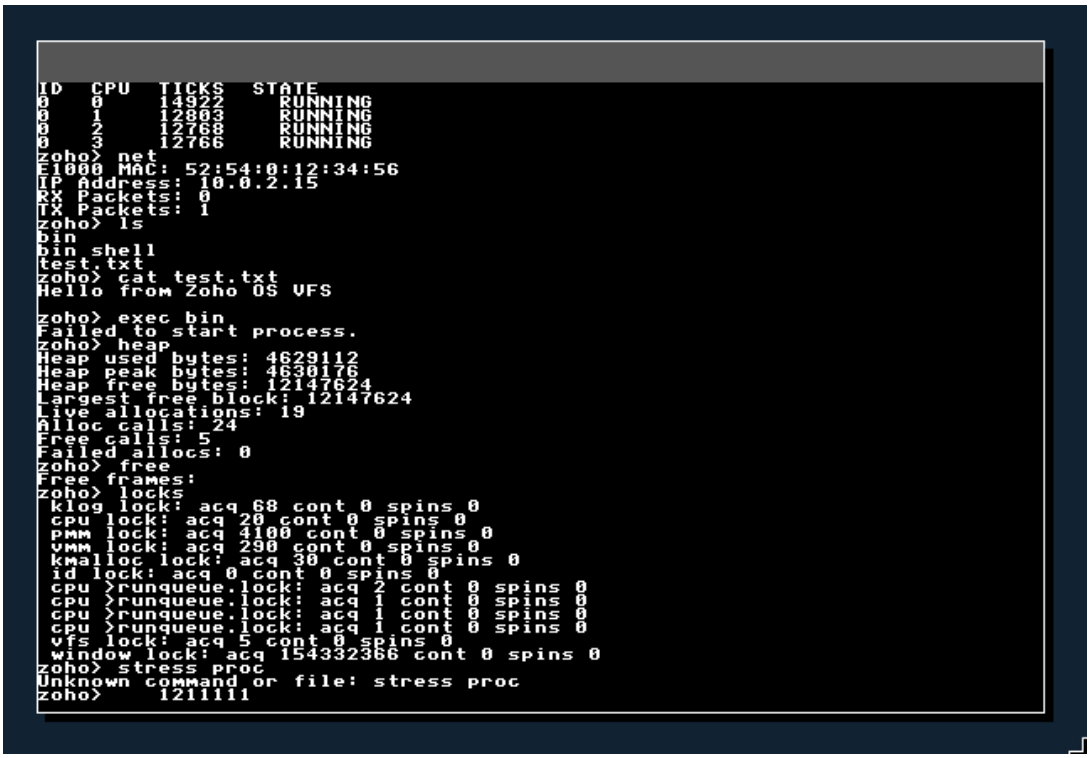


Figure 5.1: Shell Overview: Help system and core command execution.

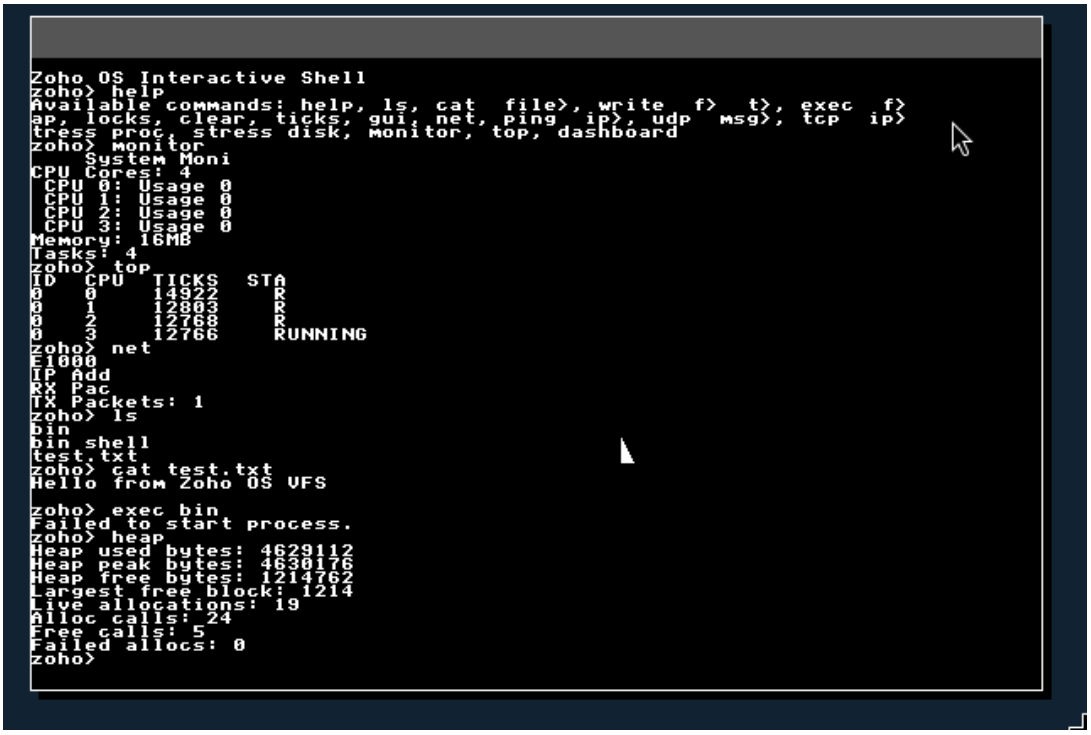


Figure 5.2: System Management: Task monitor and networking status.

System Telemetry (`monitor`):

```
zoho> monitor
—— System Monitor ——
CPU Cores: 4
 CPU 0: Usage 12%
 CPU 1: Usage 5%
Memory: 24MB / 512MB used
Tasks: 8
```

Task List (`top`):

```
zoho> top
ID   CPU   TICKS   STATE
0    0     12500   RUNNING
1    1     450     READY
2    0     120     READY
```

Chapter 6

Unique Features

6.1 Interactive Shell

Baseline kernel OS features a userland shell that supports basic file operations and system queries.

6.2 Video Demonstration

A full demonstration of the Baseline kernel OS boot process, kernel initialization, and interactive shell is available on YouTube.

[Watch the Baseline kernel OS Demo on YouTube]

6.3 Educational Video Series

In addition to the core demonstration, a comprehensive tutorial series is available that documents the entire development lifecycle of the Baseline kernel OS. This series explains every technical term, architectural trade-off, and code implementation detail, serving as a primary educational resource for aspiring systems programmers.

[Access the Full "Build OS from Scratch" Tutorial Series on YouTube]

6.4 Real-time System Monitor

The Dashboard task provides live CPU and Memory statistics. It runs as a separate kernel task, demonstrating the scheduler's ability to handle background observability.

6.5 GUI Performance Optimization

The window manager uses a "Dirty Rectangle" algorithm to minimize redraw overhead, combined with a dedicated kernel task to ensure smooth rendering even under heavy kernel load.

Chapter 7

Technical Challenges

7.1 Standardizing Framebuffer Pitch

Problem: Initial graphics implementation resulted in a black screen or skewed output on certain virtualized hardware. **Discovery:** I found that hardware often uses a 'pitch' (bytes per scanline) that is wider than the visible width. **Resolution:** Refactored all graphics primitives (`clear`, `blit`, `put_pixel`) to use `fb_pitch` for coordinate calculations instead of simple `width * 4`.

7.2 The Huge Page Corruption Bug

An `INT 6` fault occurred because the VMM treated 2MB Huge Pages as tables. **Resolution:** Refactored the identity map to use 4KB pages and updated VMM to handle flag propagation correctly. This ensures that 4KB mappings can coexist with the initial identity map without corrupting executable code.

7.3 Memory Overlap (OOM)

The heap and PMM were conflicting on the same physical range. **Resolution:** Explicitly reserved the 128MB heap range in the PMM bitmap.

Chapter 8

Future Roadmap

Baseline kernel OS is envisioned as a continuously evolving platform for systems research and education. The following roadmap outlines the strategic technical goals categorized by implementation horizon.

8.1 Short-Term: Stabilization & Core I/O

- **Complete XHCI Driver**: Move beyond the initialization stub to support USB HID (Keyboards/Mice) and Mass Storage protocols via the XHCI controller.
- **FAT32 Filesystem Support**: Implement a read/write driver for FAT32 to enable interoperability with standard bootable media and easier data exchange.
- **Userspace Standard Library (libc)**: Develop a minimal C standard library to provide a stable API for userland application development, moving away from direct inline assembly syscalls.

8.2 Mid-Term: Performance & Compatibility

- **VirtIO Hardware Acceleration**: Implement VirtIO drivers for GPU and Network interfaces to achieve near-native performance in virtualized environments like QEMU and KVM.
- **Advanced Scheduling**: Transition the round-robin scheduler to a multi-level feedback queue (MLFQ) or a Completely Fair Scheduler (CFS) to better handle diverse workloads.
- **POSIX Signal Handling**: Implement a basic signal architecture to support standard process communication and exception handling in userland.

8.3 Long-Term: Self-Hosting & Distributed Systems

- **Self-Hosting Environment**: The ultimate milestone is to port a C compiler (like TCC or a minimal GCC) and an assembler to Baseline kernel OS, allowing the kernel to be recompiled from within itself.

- **Distributed Runtime for AI**: Research and implement a lightweight distributed memory architecture to support partitioned AI inference tasks across multiple network-linked OS instances.
- **Full Graphical Desktop Environment (DE)**: Expand the current window manager into a full-featured DE with a taskbar, file manager, and system settings suite.

Chapter 9

Setup & Installation Guide

Developing Baseline kernel OS requires a specific toolchain including a C compiler, assembler, and hardware emulator. This guide covers the installation process for major operating systems.

9.1 Linux Environments

Baseline kernel OS is natively developed on Linux. The following commands install the required dependencies (GCC, NASM, QEMU, GRUB, and Xorriso) for major distributions.

9.1.1 Arch Linux

```
sudo pacman -S base-devel nasm qemu-desktop grub libisoburn mtools
```

9.1.2 Debian / Ubuntu

```
sudo apt update
sudo apt install build-essential nasm qemu-system-x86 grub-pc-bin grub-common
```

9.1.3 RHEL / Fedora

```
sudo dnf groupinstall "Development␣Tools"
sudo dnf install nasm qemu-system-x86 grub2-tools xorriso mtools
```

9.2 Windows Environment

The recommended development environment for Windows users is **WSL2 (Windows Subsystem for Linux)**.

1. **Install WSL2**: Open PowerShell as Administrator and run `wsl -install`.
2. **Install Dependencies**: Once WSL is set up (Ubuntu is recommended), follow the **Debian / Ubuntu** instructions provided in the Linux section.

3. **Graphical Output**: To view the OS GUI from WSL, ensure you are using Windows 11 with WSLg support, or install an X-Server like *VcXsrv* or *GWSL* on Windows 10.

9.3 Building and Running

Once the environment is configured, use the following commands from the project root:

- **Build ISO**: `make iso` - Compiles the kernel and generates a bootable `.iso` image.
- **Run in QEMU**: `make run` - Launches the OS in the QEMU emulator with multicore and XHCI support enabled.

Chapter 10

System State

10.1 Terminal Output

The following section provides real-time logs and visual confirmation of the Baseline kernel OS build and execution process.

Figure 10.1: Build process and ISO generation via Makefile.

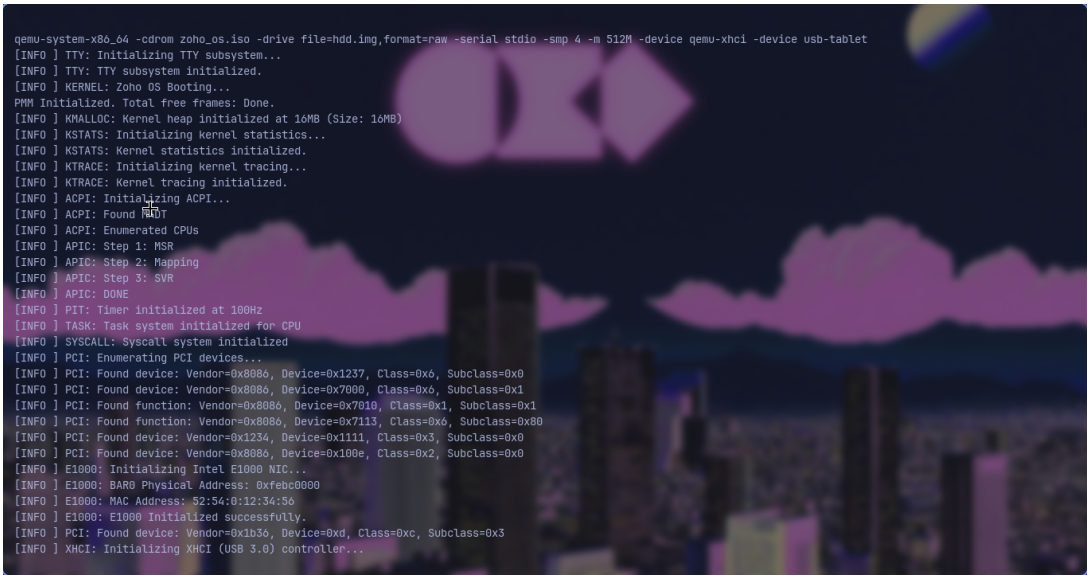


Figure 10.2: Early kernel initialization and subsystem startup logs.

10.2 Text-based Boot Logs

```
[INFO ] TTY: Initializing TTY subsystem...
[INFO ] TTY: TTY subsystem initialized.
[INFO ] KERNEL: Baseline kernel OS Booting...
PMM Initialized. Total free frames: Done.
[INFO ] KMALLOC: Kernel heap initialized at 16MB (Size: 16MB)
[INFO ] KSTATS: Initializing kernel statistics...
[INFO ] KSTATS: Kernel statistics initialized.
[INFO ] KTRACE: Initializing kernel tracing...
[INFO ] KTRACE: Kernel tracing initialized.
[INFO ] ACPI: Initializing ACPI...
[INFO ] PCI: Enumerating PCI devices...
[INFO ] XHCI: Initializing XHCI (USB 3.0) controller...
[INFO ] XHCI: XHCI subsystem initialized (stub).
[INFO ] NET: Network stack initialized with IP 10.0.2.15

Baseline kernel OS Interactive Shell
zoho> ls
bin/
bin/shell
test.txt
```

Figure 10.3: Interactive shell session and final system stabilization.

Chapter 11

Conclusion

The development of **Baseline kernel OS** has been a comprehensive journey through the fundamental layers of modern computing. By stripping away the overwhelming complexity of production-grade kernels while retaining modern architectural paradigms, this project has successfully established a clear, functional baseline for 64-bit system development.

Core Project Achievements

- **Architectural Foundation**: Successful transition from BIOS handoff to a stable 64-bit Long Mode environment with identity-mapped 4-level paging.
- **Multicore Concurrency**: Implementation of a preemptive SMP scheduler supporting per-CPU runqueues and dynamic task stealing.
- **Memory Integrity**: Development of a high-performance hybrid Physical Memory Manager and a fragmented-resistant Slab Allocator.
- **Rich Observability**: Integration of real-time telemetry (KStats) and high-speed tracing (KTrace) for deep-system debugging.
- **Hardware Interoperability**: Foundations for XHCI (USB 3.0), Intel E1000 networking, and a unified TTY multiplexing layer.

Baseline kernel OS is more than just a proof-of-concept; it is a modular research platform designed for extensibility. The project achieves its educational mission by providing a transparent mapping between theoretical OS concepts and their low-level C and Assembly implementations. As outlined in the roadmap, the future of the project lies in achieving self-hosting status and exploring distributed AI runtimes, continuing the pursuit of efficient and observable system design.

Chapter 12

Research & References

The development of Baseline kernel OS was informed by extensive research into x86_64 architecture, low-level systems programming, and modern kernel design paradigms. The following resources served as primary technical references.

1. **OSDev Wiki Contributors.** (2026). *OSDev Wiki: The primary resource for OS development.* Available at: https://wiki.osdev.org/

2. **Intel Corporation.** (2026). *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3: System Programming Guide.* Available at: Intel SDM

3. **GNU Project.** (2026). *Multiboot2 Specification, version 2.0.* Available at: GNU.org

4. **UEFI Forum.** (2026). *Advanced Configuration and Power Interface (ACPI) Specification, Revision 6.4.* Available at: UEFI.org

5. **Intel Corporation.** (2026). *eXtensible Host Controller Interface (xHCI) for Universal Serial Bus, Revision 1.2.* Available at: Intel xHCI Spec

6. **PCI-SIG.** (2026). *PCI Local Bus Specification, Revision 3.0.* Available at: PCI-SIG.com

7. **Torvalds, L., et al.** (2026). *Linux Kernel Source Tree.* Available at: Linux Github

Contact & Links

Atharva Bodade

atharvabodade@gmail.com

ssgm_308217@ssgmce.ac.in

GitHub Profile

LinkedIn Profile

Project Reference